

MicroTechnics

ФОРУМ

STM32

ЭЛЕКТРОНИКА

Микроконтроллер STM32 и Bootloader. Пример реализации.



Aveal

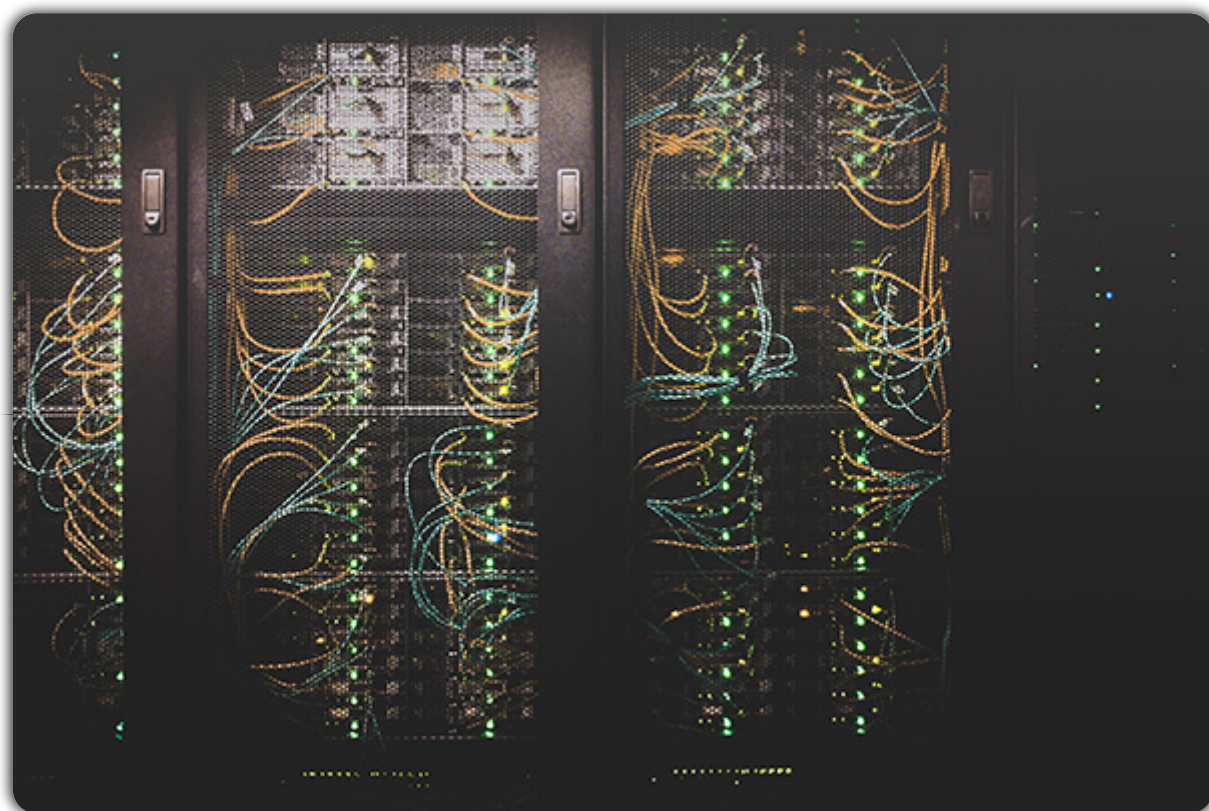
20.02.2015



153



STM32



153 Продолжаем обсуждать bootloader и сегодня, как я и обещал, создадим свой собственный загрузчик и попробуем его в деле. Все теоретические аспекты мы рассмотрели в предыдущей статье ([ссылка](#)), так что сейчас только практика и ничего кроме практики 👍

Итак, для начала разберемся с постановкой задачи. В качестве электронной

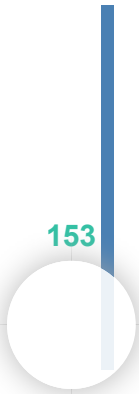
MicroTechnics

будем создавать для семейства микроконтроллеров STM32-10x. Наш bootloader будет работать с SD-картой, то есть при обнаружении файла прошивки на подключенной к плате карте, загрузчик должен будем выполнить перепрошивку контроллера. В качестве среды разработки я в этот раз буду использовать IAR. И, конечно же, в конце статьи я обязательно выложу полные проекты. Со вступлением на этом закончиваем, переходим к делу.

Определим нужные адреса, а также параметры Flash-памяти. В моем контроллере памяти 512 кБ - 256 страниц по 2 кБ. Кроме того, мы выделим отдельную страницу для хранения специального ключа-флага. Зачем он нужен? А для того, чтобы сохранить информацию при перезапуске контроллера. Перед переходом по адресу основной программы нам нужно сбросить всю периферию в начальное состояние, и это проще всего сделать просто перезапустив микроконтроллер.

Таким образом, если bootloader обнаружил прошивку и запрограммировал контроллер, он запишет в специальное место во Flash произвольное, заранее выбранное, значение и перезапустит контроллер. При входе в функцию *main()* после ресета контроллер считывает значение "ключа" из Flash-памяти, и если там записано нужное значение, то значит программа была прошита и можно выполнять переход, а если этого значения там нет, то значит перепрошивки не было, и bootloader продолжит сканировать SD-карту. Проще говоря, при включении микроконтроллер будет считывать значение из памяти, и если оно совпадет с ключом, то произойдет переход на адрес основной программы, а если нет, то bootloader продолжит просматривать карту на предмет файла прошивки.

В нашем примере в качестве ключа будет выступать значение 0xAAAA5555, и храниться ключ будет в 19-ой странице Flash-памяти:



```
1. // Bootloader key configuration
2. #define BOOTLOADER_KEY_START_ADDRESS
3. #define BOOTLOADER_KEY_PAGE_NUMBER
4. #define BOOTLOADER_KEY_VALUE
5.
6. // Flash configuration
7. #define MAIN_PROGRAM_START_ADDRESS
8. #define MAIN_PROGRAM_PAGE_NUMBER
9. #define NUM_OF_PAGES
10. #define FLASH_PAGE_SIZE
```

Основная же программа, как видите, будет прошиваться начиная с 20-ой страницы Flash. Теперь определим нужные нам функции для сохранения и

MicroTechnics

```

1.  /*****
2.  // Function          SetKey()
3.  // Description       Sets bootloader key
4.  // Parameters        None
5.  // RetVal            None
6.  *****/
7.  void SetKey()
8.  {
9.      FLASH_Unlock();
10.     FLASH_ProgramWord(BOOTLOADER_KEY_START_ADDRESS, BOOTLOADER_
11.     FLASH_Lock();
12. } // End of SetKey()
13.
14.
15. /*****
16. // Function          ResetKey()
17. // Description       Resets bootloader key
18. // Parameters        None
19. // RetVal            None
20. *****/
21. void ResetKey()
22. {
23.     FLASH_Unlock();
24.     FLASH_ErasePage(BOOTLOADER_KEY_START_ADDRESS);
25.     FLASH_Lock();
26. } // End of ResetKey()
27.
28.
29. /*****
30. // Function          ReadKey()
31. // Description       Reads bootloader key value
32. // Parameters        None
33. // RetVal            None
34. *****/
35. uint32_t ReadKey()
36. {
37.     return (*((__IO uint32_t*) BOOTLOADER_KEY_START_ADDRESS));
38. } // End of ReadKey()
39.
40.
41. /*****

```

Готово! Без лишних остановок двигаемся дальше. И на очереди функция `main()`. В самом начале проверяем ключ, и если он совпал совершаем переход:

153

```

1.  if (ReadKey() == BOOTLOADER_KEY_VALUE)
2.  {
3.      ResetKey();
4.      __disable_irq();
5.      NVIC_SetVectorTable(NVIC_VectTab_FLASH, MAIN_PROGRAM_START_
6.
7.      jumpAddress = (*((__IO uint32_t*) (MAIN_PROGRAM_START_ADDRES
8.      Jump_To_Application = (pFunction) jumpAddress;

```

MicroTechnics

11. }

Переход осуществляется следующим образом. Первым делом сбрасываем ключ и выключаем прерывания. Затем переносим таблицу векторов прерываний на адреса, соответствующие основной программе. А непосредственно переход заключается в последующих четырех строках. Важным моментом является то, что мы берем адрес `(MAIN_PROGRAM_START_ADDRESS + 4)`, это связано с тем, что по адресу `MAIN_PROGRAM_START_ADDRESS` сначала записывается таблица векторов прерываний.

Идем дальше и тут начинается самое интересное - если ключ не совпал, то мы начинаем проверять SD-карту на наличие файла прошивки. И для начала инициализируем интерфейс SDIO:

```
1.  NVIC_InitTypeDef NVIC_InitStructure;
2.  SD_Init();
3.
4.  // SDIO Interrupt ENABLE
5.  NVIC_InitStructure.NVIC_IRQChannel = SDIO_IRQn;
6.  NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 0;
7.  NVIC_InitStructure.NVIC_IRQChannelSubPriority = 0;
8.  NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;
9.  NVIC_Init(&NVIC_InitStructure);
10.
11. // DMA2 Channel4 Interrupt ENABLE
12. NVIC_InitStructure.NVIC_IRQChannel = DMA2_Channel4_5_IRQn;
13. NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 1;
14. NVIC_Init(&NVIC_InitStructure);
15. SD_GetCardInfo(&SDCard);
16. SD_SelectDeselect((uint32_t)(SDCard.RCA << 16));
```

Работу с картами памяти мы уже много раз обсуждали, поэтому на этом не будем подробно останавливаться. А дальше мы проверяем, есть ли на карте файл прошивки (зададим для него имя - *program.bin*):

153

```
1.  FATFS SDFS;
2.  FIL program;
3.
4.  if(f_mount(&SDFS, "0", 1) == FR_OK)
5.  {
6.      uint8_t path[12] = "program.bin";
7.      path[11] = '\\0';
8.
9.      result = f_open(&program, (char*)path, FA_READ);
10.
11.      if (result == FR_OK)
12.      {
```

MicroTechnics

И если нужный нам файл найден, то следует произвести программирование. Реализуем это, и для начала разблокируем Flash-память:

```
1.  FLASH_Unlock();
```

Теперь нам необходимо произвести очистку всех страниц памяти, соответствующих основной программе. В нашем случае это страницы с 20-ой до последней, 256-ой:

```
1.  for(uint8_t i = 0; i < (NUM_OF_PAGES - MAIN_PROGRAM_PAGE_NUMBER); i++)
2.  {
3.      FLASH_ErasePage(MAIN_PROGRAM_START_ADDRESS + i * FLASH_PAGE_SIZE);
4.  }
```

Само программирование мы будем осуществлять порциями по 512 байт, то есть считываем из файла 512 байт, записываем их во Flash, затем считываем следующие 512 байт и так далее... При этом будем сохранять в переменной количество уже записанных байт и сравнивать это значение с общим числом байт. Если осталось записать больше 512 байт, то считываем их из файла и записываем. Как только нам останется записать меньше 512 байт, мы считываем оставшееся количество и записываем во Flash эти последние байты. В программе это выглядит следующим образом:

```
1.  programBytesToRead = program.fsize;
2.  programBytesCounter = 0;
3.  currentAddress = MAIN_PROGRAM_START_ADDRESS;
4.
5.  while ((programBytesToRead - programBytesCounter) >= 512)
6.  {
7.      fread(&program, readBuffer, 512, &readBytes);
8.      programBytesCounter += 512;
9.
10.     for (uint32_t i = 0; i < 512; i += 4)
11.     {
12.         FLASH_ProgramWord(currentAddress, *(uint32_t*)&readBuffer[i]);
13.         currentAddress += 4;
14.     }
15. }
16.
17. if (programBytesToRead != programBytesCounter)
18. {
19.     fread(&program, readBuffer, (programBytesToRead - programBytesCounter), &readBytes);
20.
21.     for (uint32_t i = 0; i < (programBytesToRead - programBytesCounter); i++)
22.     {
23.         FLASH_ProgramWord(currentAddress, *(uint32_t*)&readBuffer[i]);
24.         currentAddress += 4;
25.     }
26. }
```

153

MicroTechnics

И после окончания этих операций не забываем заблокировать Flash и закрыть файл:

```
1. FLASH_Lock();
2. f_close(&program);
```

Кроме того, удаляем файл прошивки с SD-карты, ведь иначе при каждом перезапуске bootloader будет находить один и тот же файл и раз за разом записывать его во Flash:

```
1. f_unlink((char*)path);
```

Отключаем прерывания, сохраняем ключ и выполняем перезапуск контроллера:

```
1. NVIC_DisableIRQ(DMA2_Channel4_5_IRQn);
2. NVIC_DisableIRQ(SDIO_IRQn);
3. ResetKey();
4. SetKey();
5. NVIC_SystemReset();
```

На этом с программированием памяти вроде бы все. А что делать, если файл не обнаружен? В этом случае мы анализируем байты, записанные по адресу основной программы и если они сигнализируют нам о том, что основная программа уже находится во Flash (то есть была туда помещена ранее), то мы делаем вывод, что она не нуждается в обновлении (ведь файл прошивки отсутствует), записываем ключ и выполняем перезапуск. А контроллер при включении, увидев ключ, просто перейдет на основную программу.

Если же основная программа отсутствует в памяти, то мы также выполняем перезапуск, но не записываем при этом ключ. Тогда контроллер, перезапустившись, начнет заново сканировать карту-памяти на наличие файла прошивки:

```
153 1. if (((*(uint32_t*)MAIN_PROGRAM_START_ADDRESS) & 0x2FFF0000) ==
2. {
3.     ResetKey();
4.     SetKey();
5.     NVIC_SystemReset();
6. }
7. else
8. {
9.     NVIC_SystemReset();
10. }
```


MicroTechnics

загрузчиком, но не заканчиваем статью. давайте создадим тестовый проект, моргающий светодиодом, для того, чтобы протестировать работу нашего бутлодера.

Итак, создаем абсолютно новый проект, и код там будет достаточно простой - инициализируем таймер и вывод, на котором находится светодиод на плате. И в прерывании по таймеру моргаем диодом. Отличие от обычных проектов тут будет в том, что в начале функции `main()` мы вызываем функцию для переноса таблицы векторов прерываний в нужное нам место:

```
1.  int main()
2.  {
3.      __set_PRIMASK(1);
4.      NVIC_SetVectorTable(NVIC_VectTab_FLASH, 0x0800A000);
5.      __set_PRIMASK(0);
6.
7.      __enable_irq();
8.
9.      initAll();
10.
11.     TIM_ITConfig(TIM4, TIM_IT_Update, ENABLE);
12.     TIM_Cmd(TIM4, ENABLE);
13.     NVIC_EnableIRQ(TIM4_IRQn);
14.
15.     while(1)
16.     {
17.     }
18.
19.     return 0;
20. }
21.
22.
23. void TIM4_IRQHandler()
24. {
25.     TIM_ClearITPendingBit(TIM4, TIM_IT_Update);
26.
27.     if (currentState == 0)
28.     {
29.         currentState = 1;
30.         GPIO_SetBits(GPIOC, GPIO_Pin_6);
31.     }
32.     else
33.     {
34.         currentState = 0;
35.         GPIO_ResetBits(GPIOC, GPIO_Pin_6);
36.     }
37. }
```

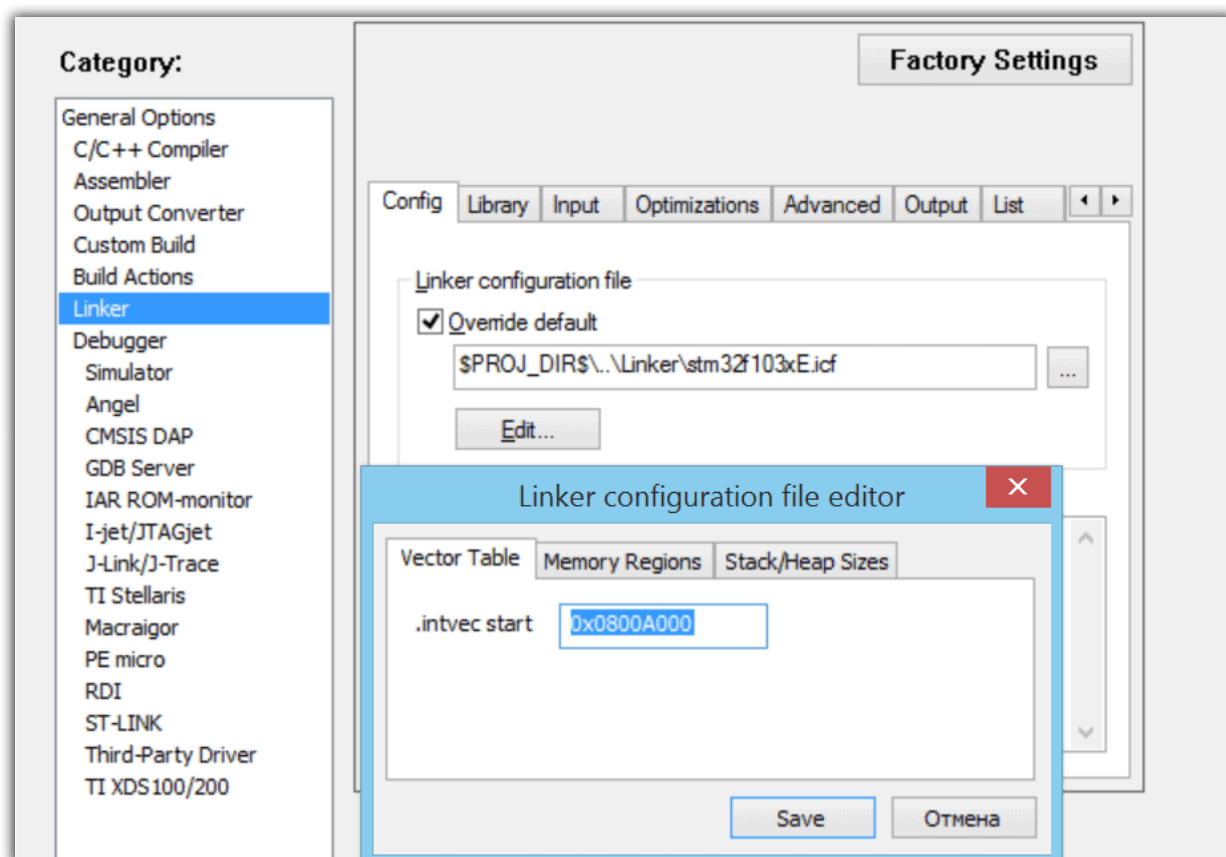
153

Этот тестовый проект я также выложу в конце статьи.

На самом деле и это еще не все, наш проект мы должны еще правильным

MicroTechnics

соответствующие нашей основной программе, то есть в данном случае адрес 0x0800A000 (пример для IAR):



Также во вкладке Memory Regions в поле ROM Start надо указать тот же адрес - 0x0800A000.

Возможно вы обратили внимание на то, что у меня используется не стандартный файл линкера, а другой (на скриншоте выставлена галочка Override default и указан путь к моему файлу). Сейчас расскажу, зачем это сделано.

Если мы будем использовать стандартный файл, входящий в состав IAR, и поменяем в нем адреса для проекта с bootloader'ом, то затем, создав новый проект для такого же контроллера, в котором bootloader не используется или используются другие адреса, мы получим ошибки, ведь файл линкера один и тот же и он уже изменен нами ранее совсем для другого проекта. Поэтому для всех проектов я копирую стандартный файл линкера к себе в папку с проектом и уже в него спокойно записываю новые адреса, поскольку этот файл будет использоваться только для этого проекта.

И вот на этом мы на сегодня заканчиваем, надеюсь статья получилась